

CRYPTOGRAPHY

Implementation Manual & Reference Guide

Lattice-Based Encryption · Hash-Based Signatures · Hybrid Model

Student Name	Tirth Somani
Enrollment No.	2023050905090529
Class	G
Branch	B.Tech (Information Technology)
Institution	GMIU (Gyan Manjari Innovative University)
Subject	Post-Quantum Cryptography Implementation
Submission Year	2025

GitHub Repository: github.com/tirthsomani/post-quantum-crypto

Table of Contents

1. Introduction to Post-Quantum Cryptography

- 1.1 What is Post-Quantum Cryptography?
- 1.2 Why Classical Cryptography is Vulnerable
- 1.3 NIST PQC Standardisation Process

2. Lattice-Based Encryption (Kyber / ML-KEM)

- 2.1 Mathematical Foundations
- 2.2 Learning With Errors (LWE) Problem
- 2.3 Algorithm Overview
- 2.4 Python Implementation
- 2.5 Security Analysis

3. Hash-Based Digital Signatures (SPHINCS+)

- 3.1 Hash Function Security
- 3.2 XMSS and SPHINCS+ Overview
- 3.3 Algorithm Design
- 3.4 Python Implementation
- 3.5 Performance Characteristics

4. Hybrid Cryptographic Model

- 4.1 Motivation for Hybrid Systems
- 4.2 TLS 1.3 + Kyber Hybrid
- 4.3 Implementation Architecture
- 4.4 Python Implementation

5. Comparative Analysis

- 5.1 Security vs Performance Trade-offs
- 5.2 Use-Case Recommendations

6. GitHub Repository Structure

- 6.1 Directory Layout
- 6.2 Running the Code

7. References

1. Introduction to Post-Quantum Cryptography

1.1 What is Post-Quantum Cryptography?

Post-Quantum Cryptography (PQC) refers to cryptographic algorithms that are believed to be secure against both classical computers and the far more powerful quantum computers. As quantum hardware continues to mature, many of the mathematical problems underpinning current cryptographic schemes — such as integer factorisation and the discrete logarithm problem — can be solved efficiently using quantum algorithms like Shor's algorithm, rendering RSA, ECC, and Diffie-Hellman obsolete.

PQC schemes are designed around mathematical problems that even quantum computers cannot solve efficiently in polynomial time. NIST completed its evaluation process in 2024, standardising CRYSTALS-Kyber (ML-KEM), CRYSTALS-Dilithium (ML-DSA), FALCON, and SPHINCS+ as the first PQC standards.

1.2 Why Classical Cryptography is Vulnerable

Classical Algorithm	Underlying Problem	Quantum Attack	Status
RSA-2048	Integer Factorisation	Shor's Algorithm	Broken
ECDSA P-256	Elliptic Curve Discrete Log	Shor's Algorithm	Broken
AES-128	Symmetric Key Security	Grover's Algorithm	Weakened (use 256-bit)
SHA-256	Collision Resistance	Grover's Algorithm	Weakened (use SHA-3)
Kyber-768	Module-LWE Problem	No efficient attack	Secure
SPHINCS+-256	Hash Function Security	Grover (mitigated)	Secure

1.3 NIST PQC Standardisation Timeline

Phase	Year	Activity
Round 1	2017–2019	69 submissions received, initial evaluation
Round 2	2019–2020	26 candidates selected for deeper analysis
Round 3	2020–2022	7 finalists + 8 alternates shortlisted
Draft Std	2022–2023	CRYSTALS-Kyber, Dilithium, FALCON, SPHINCS+ selected
Final Std	2024	FIPS 203/204/205 published — first official PQC standards

2. Lattice-Based Encryption (Kyber / ML-KEM)

2.1 Mathematical Foundations

Lattice-based cryptography builds on the computational hardness of problems in high-dimensional integer lattices. A lattice L is a discrete additive subgroup of $\mathbb{R}^n$, defined by a basis matrix $B \in \mathbb{Z}^{(n \times n)}$. The two fundamental hard problems are:

- **Shortest Vector Problem (SVP):** Find the shortest non-zero vector in a lattice. No polynomial-time classical or quantum algorithm is known.
- **Closest Vector Problem (CVP):** Given a lattice and a target point, find the lattice point closest to the target.

2.2 Learning With Errors (LWE) Problem

LWE Problem (Regev, 2005)

Given a matrix $A \in \mathbb{Z}_q^{(m \times n)}$, secret vector $s \in \mathbb{Z}_q^n$, and small error $e \in \chi^m$ (drawn from a Gaussian distribution), the LWE problem asks to distinguish $(A, As + e)$ from (A, u) where u is uniformly random over $\mathbb{Z}_q^m$. No quantum algorithm can solve this in sub-exponential time.

Kyber uses the **Module-LWE (MLWE)** variant for improved efficiency. The module structure allows the use of ring arithmetic, dramatically reducing key sizes while maintaining security. Kyber-768 targets NIST security level 3 (comparable to AES-192) with 1,184-byte public keys.

2.3 Algorithm Overview

Operation	Algorithm Steps	Output
Key Generation	Sample A from seed; sample s, e from χ ; compute $t = As + e$	Public key (A, t) , Secret key s
Encapsulation	Sample r, e_1, e_2 from χ ; compute $u = A^T r + e_1$; $v = t^T r + e_2 + \text{encode}(m)$	Ciphertext (u, v) , Shared secret K
Decapsulation	Compute $m' = \text{decode}(v - s^T u)$; derive K from m'	Shared secret K

2.4 Python Implementation

The following implementation demonstrates Kyber-like key encapsulation using the `pqcrypto` and `kyber-py` libraries. For production use, always prefer NIST-validated implementations.

● kyber_kem.py

[illegible]

2.5 Security Analysis

Parameter Set	Security Level	Public Key	Secret Key	Ciphertext
Kyber-512	NIST Level 1 (AES-128)	800 B	1632 B	768 B
Kyber-768	NIST Level 3 (AES-192)	1184 B	2400 B	1088 B
Kyber-1024	NIST Level 5 (AES-256)	1568 B	3168 B	1568 B

3. Hash-Based Digital Signatures (SPHINCS+)

3.1 Hash Function Security

Hash-based signature schemes derive their security entirely from the properties of cryptographic hash functions: pre-image resistance, second pre-image resistance, and collision resistance. Even Grover's quantum algorithm only halves the effective security level, meaning a 256-bit hash provides 128-bit post-quantum security — fully compliant with NIST requirements.

3.2 XMSS and SPHINCS+ Overview

XMSS (eXtended Merkle Signature Scheme, RFC 8391) is a stateful scheme — signers must track used one-time keys. **SPHINCS+** (FIPS 205) solves this by using a hypertree of Merkle trees with few-time signing at each level, producing a stateless signature scheme with strong security proofs based solely on hash security.

3.3 Algorithm Design

SPHINCS+ uses the following layered structure:

- **WOTS+ (Winternitz OTS)**: One-time signature scheme at the leaf level.
- **FORS (Forest of Random Subsets)**: Few-time signature scheme for message digests.
- **HT (Hypertree)**: Multi-layer Merkle tree for signing FORS public keys.
- **Top-level Merkle tree**: Root defines the long-term public key.

3.4 Python Implementation

● sphincs_signer.py — Class Definition

```
# File: hash_signatures/sphincs_signer.py
# SPHINCS+ Digital Signatures (FIPS 205)

from sphincs import SphincsPlus # pip install sphincs-py
import hashlib, os

class SphincsSigner:
    VARIANTS = {
        'fast' : 'sphincs-shake-256f',
        'small' : 'sphincs-shake-256s',
    }

    def __init__(self, variant='small'):
        self.scheme = SphincsPlus(self.VARIANTS[variant])

    def keygen(self):
        pk, sk = self.scheme.keygen()
        return pk, sk

    def sign(self, sk, message: bytes) -> bytes:
        if isinstance(message, str):
            message = message.encode('utf-8')
        return self.scheme.sign(message, sk)

    def verify(self, pk, message: bytes, sig: bytes) -> bool:
        if isinstance(message, str):
            message = message.encode('utf-8')
        try:
            return self.scheme.verify(message, sig, pk)
        except Exception:
            return False
```

- sphincs_signer.py — Demo

```
# Demo
if __name__ == '__main__':
    signer = SphincsSigner(variant='small')
    pk, sk = signer.keygen()
    print(f'Public key : {len(pk)} bytes')
    print(f'Secret key : {len(sk)} bytes')

    msg = b'GMIU PQC Assignment 2025 - Tirth Somani'
    sig = signer.sign(sk, msg)
    print(f'Signature : {len(sig)} bytes')

    ok = signer.verify(pk, msg, sig)
    print(f'Valid signature: {ok}')

    tampered = b'Tampered message'
    ok2 = signer.verify(pk, tampered, sig)
    print(f'Tampered msg accepted: {ok2}') # Must be False
```

3.5 Performance Characteristics

Scheme	Key Gen	Sign	Verify	Sig Size	Security
SPHINCS+-128s	~1 ms	~600 ms	~10 ms	7,856 B	Level 1
SPHINCS+-192s	~2 ms	~1.2 s	~18 ms	16,224 B	Level 3
SPHINCS+-256s	~3 ms	~2.5 s	~30 ms	29,792 B	Level 5
CRYSTALS-Dilithium3	~0.1 ms	~0.2 ms	~0.1 ms	3,293 B	Level 3
RSA-2048 (classical)	~100 ms	~5 ms	~0.5 ms	256 B	Broken PQ

4. Hybrid Cryptographic Model

4.1 Motivation for Hybrid Systems

A hybrid model combines a classical algorithm (e.g., X25519 ECDH) with a post-quantum algorithm (e.g., Kyber-768) such that security is maintained as long as at least one of the two components remains secure. This is the recommended migration strategy by NIST, ANSSI (France), and BSI (Germany) because it guards against both classical and quantum adversaries during the transition period.

Industry Adoption

Google, Cloudflare, and Mozilla have already deployed X25519+Kyber hybrid key exchange in Chrome and Firefox (TLS 1.3). IETF RFC 9180 and RFC 8446 amendments cover hybrid KEM standardisation.

4.2 TLS 1.3 + Kyber Hybrid Architecture

Step	Classical (X25519)	Post-Quantum (Kyber-768)	Combined Output
1 — Key Exchange	Client/Server X25519 ECDH → shared secret K_classical	Server generates Kyber KEM keypair; client encapsulates → K_pq	K_hybrid = KDF(K_classical K_pq)
2 — Authentication	RSA/ECDSA certificate (existing PKI)	Optional ML-DSA (Dilithium) co-signature	Dual-signed TLS handshake
3 — Symmetric Encryption	AES-256-GCM (128-bit quantum security)	ChaCha20-Poly1305 (alternative)	Forward secret channel

4.3 Python Implementation — Hybrid KEM

● hybrid_kem.py — Class Definition

```
# File: hybrid_model/hybrid_kem.py
# Hybrid Key Exchange: X25519 + Kyber-768

import os, hashlib
from cryptography.hazmat.primitives.asymmetric.x25519 import X25519PrivateKey
from cryptography.hazmat.primitives.hashes import SHA256
from cryptography.hazmat.primitives.kdf.hkdf import HKDF
from kyber_py.kyber import Kyber768

class HybridKEM:
    def classical_keygen(self):
        sk = X25519PrivateKey.generate()
        return sk, sk.public_key()

    def classical_exchange(self, my_sk, peer_pk):
        return my_sk.exchange(peer_pk) # 32 bytes

    def pq_keygen(self):
        return Kyber768.keygen() # (pk, sk)

    def pq_encapsulate(self, pk):
        return Kyber768.enc(pk) # (ct, ss)

    def pq_decapsulate(self, sk, ct):
        return Kyber768.dec(sk, ct)

    def combine(self, classical_ss: bytes, pq_ss: bytes) -> bytes:
        hkdf = HKDF(algorithm=SHA256(), length=32,
        salt=None, info=b'hybrid-kem-v1')
        return hkdf.derive(classical_ss + pq_ss)
```

- `hybrid_kem.py` — Demo

```
# Demo
if __name__ == '__main__':
    kem = HybridKEM()

    # Server side
    srv_x_sk, srv_x_pk = kem.classical_keygen()
    srv_k_pk, srv_k_sk = kem.pq_keygen()

    # Client side
    cli_x_sk, cli_x_pk = kem.classical_keygen()
    k_ct, pq_ss_c = kem.pq_encapsulate(srv_k_pk)
    x_ss_c = kem.classical_exchange(cli_x_sk, srv_x_pk)
    key_client = kem.combine(x_ss_c, pq_ss_c)

    # Server decapsulation
    pq_ss_s = kem.pq_decapsulate(srv_k_sk, k_ct)
    x_ss_s = kem.classical_exchange(srv_x_sk, cli_x_pk)
    key_server = kem.combine(x_ss_s, pq_ss_s)

    print('Keys match:', key_client == key_server)
    print('Hybrid key:', key_client.hex())
```

5. Comparative Analysis

5.1 Security vs Performance Trade-offs

Scheme	Type	PQ Secure	Sig/CT Size	Speed	Maturity
RSA-2048	Encryption	No	256 B	Fast	Legacy
ECDSA P-256	Signature	No	72 B	Fast	Legacy
Kyber-768	KEM	Yes	1088 B	Fast	FIPS 203
Dilithium3	Signature	Yes	3293 B	Fast	FIPS 204
FALCON-1024	Signature	Yes	1280 B	Medium	FIPS 206
SPHINCS+-256s	Signature	Yes	29792 B	Slow	FIPS 205
X25519+Kyber	Hybrid KEM	Yes*	+1088 B	Fast	IETF Draft

* Secure if either classical or PQ component remains secure.

5.2 Use-Case Recommendations

Use Case	Recommended Scheme	Reason
TLS Key Exchange	Kyber-768 (Hybrid)	Fast, small ciphertext, IETF support
Software Code Signing	Dilithium3 or FALCON	Good speed/size balance
Long-lived Certificates	SPHINCS+-256s	No state, pure hash security
IoT / Embedded Systems	Kyber-512 + Dilithium2	Minimal memory footprint
High-Assurance Gov. Systems	SPHINCS+-256s	Conservative, no algebraic assumptions
Blockchain Wallets	FALCON-512	Smallest PQ signature scheme

6. GitHub Repository Structure

6.1 Directory Layout

● Repository Structure

```
post-quantum-crypto/
├── README.md # Project overview & setup
├── requirements.txt # Python dependencies
├──
├── lattice_encryption/
│   ├── kyber_kem.py # Kyber-768 key encapsulation
│   ├── lwe_demo.py # Learning With Errors demo
│   └── benchmarks.py # Performance benchmarking
│   └──
│       ├── hash_signatures/
│       │   ├── sphincs_signer.py # SPHINCS+ sign & verify
│       │   ├── xmss_demo.py # XMSS stateful demo
│       │   └── hash_security.py # Hash function comparison
│       │   └──
│       │       ├── hybrid_model/
│       │       │   ├── hybrid_kem.py # X25519 + Kyber hybrid
│       │       │   ├── tls_simulation.py # TLS 1.3 hybrid handshake
│       │       │   └── key_combination.py # HKDF key derivation
│       │       │   └──
│       │       │       ├── tests/
│       │       │       │   ├── test_kyber.py
│       │       │       │   ├── test_sphincs.py
│       │       │       │   └── test_hybrid.py
│       │       │       │   └──
│       │       │       │       ├── docs/
│       │       │       │       │   ├── PQC_Manual_Tirth_Somani.pdf # This manual
│       │       │       │       │   └──
│       │       │       │       │       ├── report/
│       │       │       │       │       │   ├── analysis.md # Written analysis
│       │       │       │       │       │   └── performance_graphs/ # Benchmark charts
```

6.2 Running the Code

● Setup & Execution

```
# 1. Clone the repository
git clone https://github.com/tirthsomani/post-quantum-crypto
cd post-quantum-crypto

# 2. Create virtual environment
python -m venv .venv
source .venv/bin/activate # Linux / macOS
.venv\Scripts\activate # Windows

# 3. Install dependencies
pip install -r requirements.txt

# 4. Run individual demos
python lattice_encryption/kyber_kem.py
python hash_signatures/sphincs_signer.py
python hybrid_model/hybrid_kem.py

# 5. Run all tests
pytest tests/ -v

# 6. Run benchmarks
python lattice_encryption/benchmarks.py
```

requirements.txt

● requirements.txt

```
kyber-py>=0.7.0
sphincs-py>=0.2.0
cryptography>=42.0.0
pycryptodome>=3.20.0
pytest>=8.0.0
numpy>=1.26.0
matplotlib>=3.8.0
```

7. References

[1]

NIST FIPS 203 — Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM / Kyber). National Institute of Standards and Technology, 2024. <https://doi.org/10.6028/NIST.FIPS.203>

[2]

NIST FIPS 205 — Stateless Hash-Based Digital Signature Standard (SLH-DSA / SPHINCS+). NIST, 2024. <https://doi.org/10.6028/NIST.FIPS.205>

[3]

Regev, O. — On Lattices, Learning with Errors, Random Linear Codes, and Cryptography. Journal of the ACM, 56(6), 2009.

[4]

Bernstein & Lange — Post-Quantum Cryptography. Nature, 549, 188–194, 2017.

[5]

RFC 8391 — XMSS: eXtended Merkle Signature Scheme. IETF, 2018.

[6]

Cloudflare Blog — The TLS Post-Quantum Experiment. <https://blog.cloudflare.com/the-tls-post-quantum-experiment/>

[7]

NIST PQC Round 3 — Submission packages for Kyber, Dilithium, FALCON, SPHINCS+. <https://csrc.nist.gov/projects/post-quantum-cryptography>

[8]

kyber-py library — Python implementation of Kyber KEM. <https://github.com/jack4818/kyber-py>

Student Declaration

I, **Tirth Somani** (Enrollment No. **2023050905090529**), B.Tech Information Technology, Class G, GMIU, hereby declare that this implementation manual and the associated GitHub repository represent my original work. All external libraries and references are duly cited above. The code has been tested and the results are reproducible.

Name: Tirth Somani	Enroll: 2023050905090529	Branch: B.Tech (I.T.) Class G	Date: 14 April 2026
--------------------	--------------------------	---------------------------------	---------------------